

**Password Capturing and HTTP Credential Analysis Using Wireshark**

**DATE : 06-08-2026**

**1. AIM**

To capture and analyse network traffic using Wireshark, identify HTTP GET and POST requests, and examine login credentials transmitted through unencrypted HTTP traffic.

**2. SOFTWARE / TOOLS REQUIRED**

- **Operating System:** Windows / Linux
- **Packet Analyzer:** Wireshark
- **Web Browser:** Google Chrome / Mozilla Firefox
- **Network Interface:** Wi-Fi / Ethernet

**3. THEORY**

Wireshark is a network packet analyzer used to capture and inspect data transmitted over a network. It can capture different types of network information, including HTTP requests, usernames, passwords, and other data when the communication is not encrypted.

Protocols such as **HTTP, FTP, and Telnet** may transmit information in plain text. Therefore, sensitive information can potentially be visible in captured packets.

HTTP forms generally use two request methods:

- **GET:** Used mainly to request information from a server. Parameters may be included in the URL.
- **POST:** Used to submit form data to a server. When HTTP is used without encryption, the submitted data may be visible in the captured packet.

**Password Capturing**

Wireshark can capture not only passwords but any type of information transmitted over the network: usernames, email addresses, personal information, etc. As long as we can capture network traffic, Wireshark can sniff passing passwords.

In sniffing can include passwords for various protocols such as HTTP, FTP, Telnet, etc. the captured data can be used to troubleshoot network problems, but can also be used maliciously to gain unauthorized access to sensitive information.

**4. PROCEDURE**

**Step 1: Starting Wireshark**

1. Open **Wireshark** on the system.
2. Select the active network interface, such as **Wi-Fi**.
3. Start packet capture.

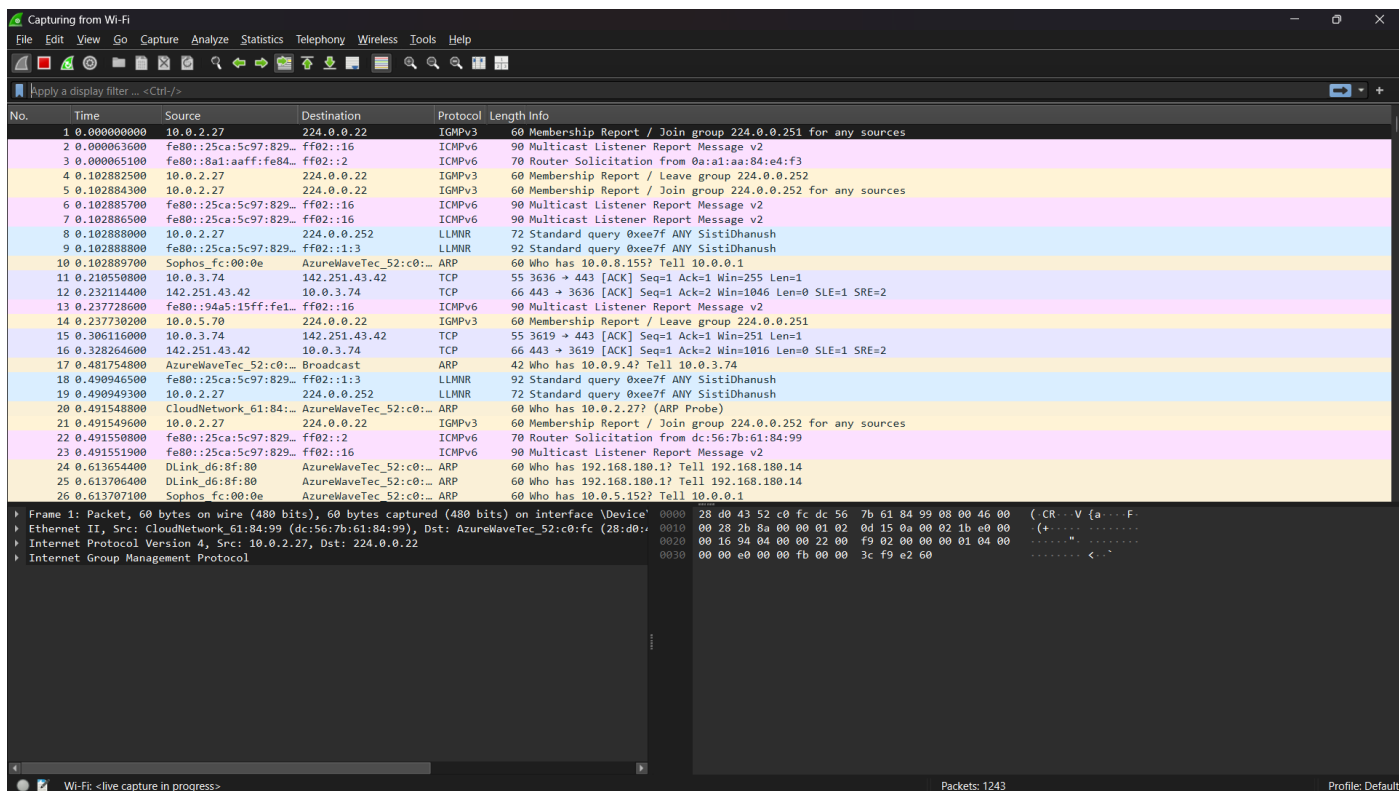


Figure 1: Wireshark Network Interface Capture

## Step 2: Generating Network Traffic

1. Open the required website in a web browser.
2. Enter the login credentials in the login form.
3. Submit the login form.



Figure 2: Login Form Submission

## Step 3: Filtering HTTP Traffic

After submitting the login form, return to Wireshark and apply the following display filter:

http

This filter displays the captured HTTP packets.

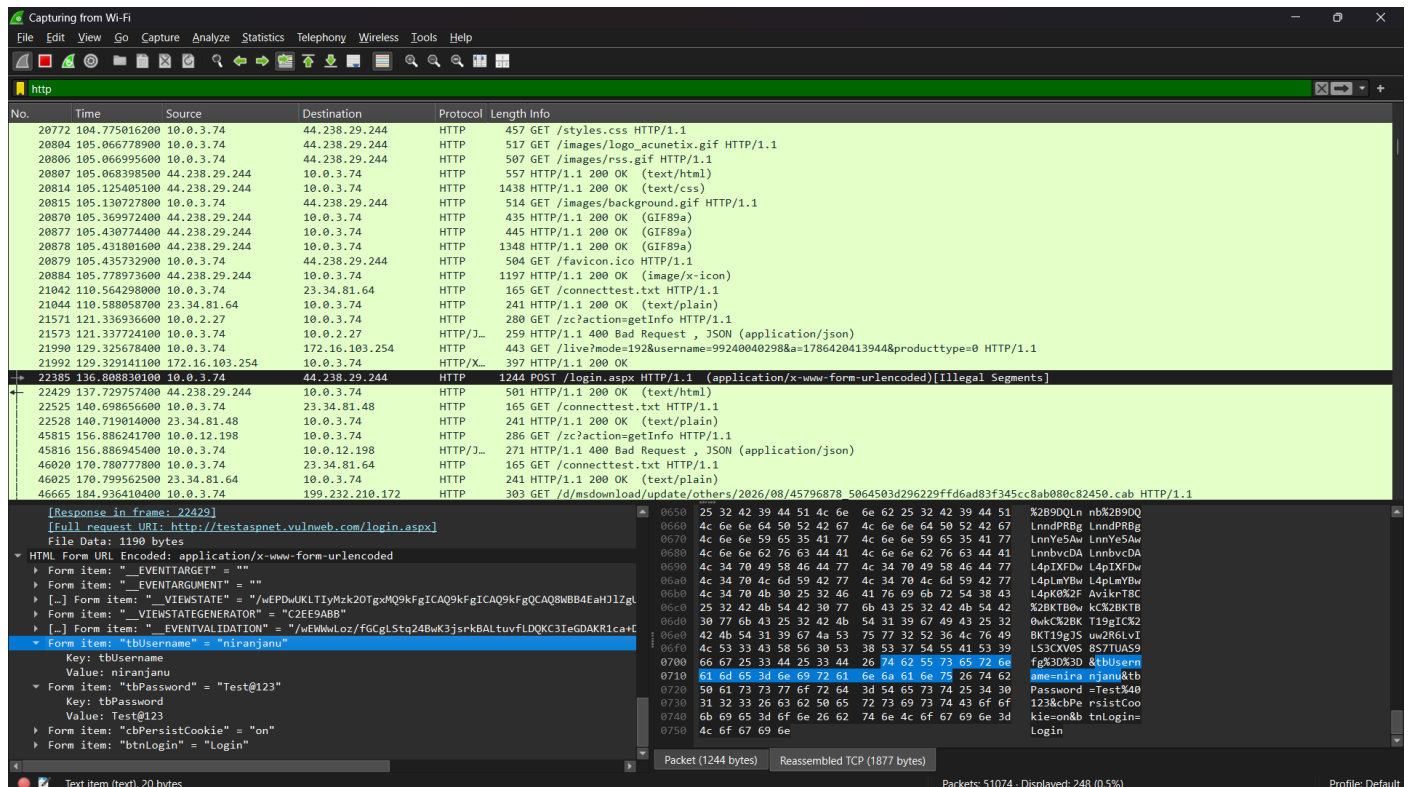
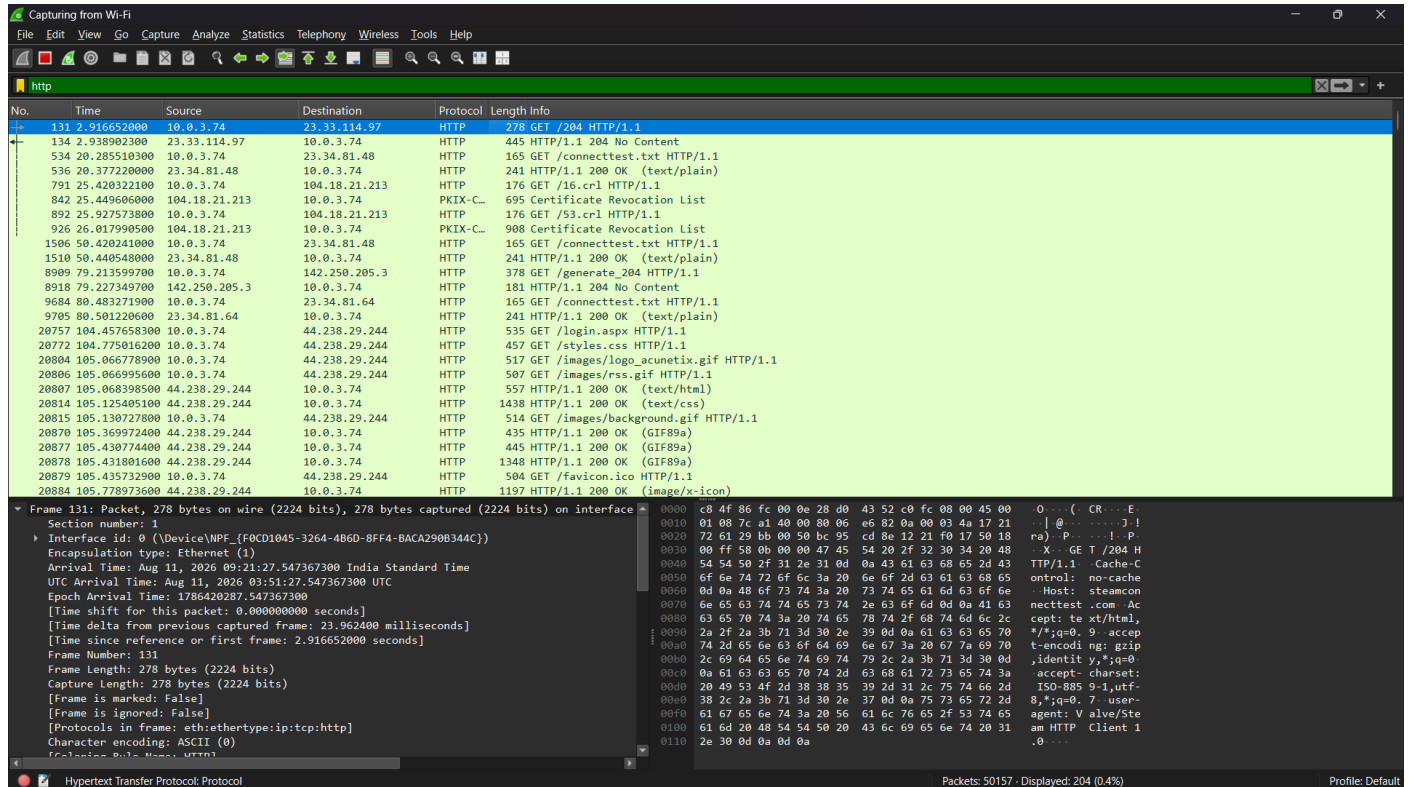


Figure 3: Applying HTTP Display Filter

## Step 4: Analyzing GET Requests

To identify HTTP GET requests, apply the following filter:

```
http.request.method == "GET"
```

The captured GET packets can be inspected to determine whether any submitted form information is present.

**Observation:** The GET requests mainly contain requests for web pages or other resources and may not contain the submitted login form data.

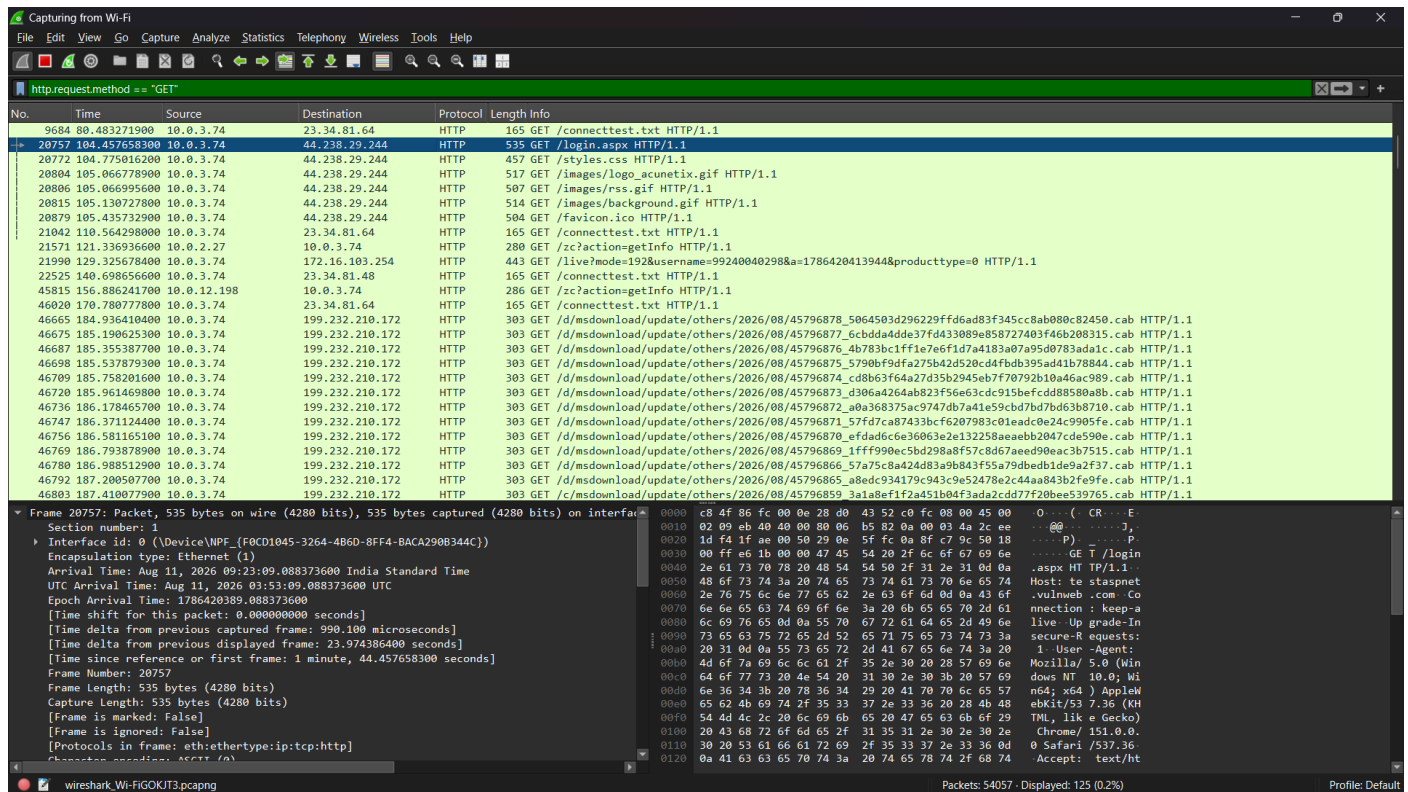


Figure 4: Analysis of HTTP GET Requests

## Step 5: Analyzing POST Requests

If the login information is not found in the GET requests, apply the following filter:

```
http.request.method == "POST"
```

Select the packet containing the submitted form data.

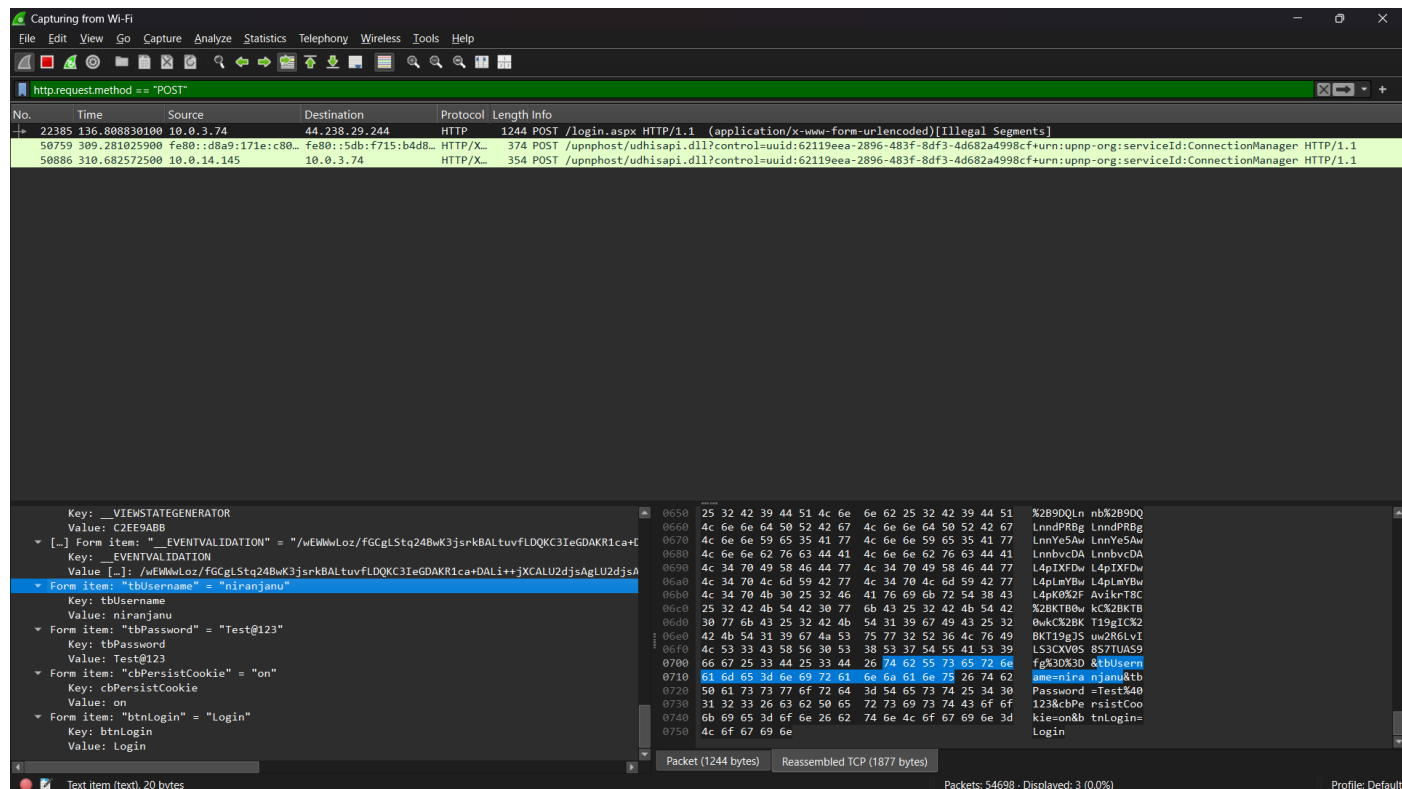


Figure 5: Analysis of HTTP POST Request

## Step 6: Inspecting Form Data

Expand the **HTML Form URL Encoded** section in the Packet Details pane.

The submitted form fields can be viewed when the website uses unencrypted HTTP communication.

### Captured Form Data:

Form item: "uname" = "niranjanu"

Form item: "pass" = "Test@123"

```
HTML Form URL Encoded: application/x-www-form-urlencoded
  Form item: "__EVENTTARGET" = ""
  Form item: "__EVENTARGUMENT" = ""
  [...] Form item: "__VIEWSTATE" = "/wEPDwUKLTiyMzk2OTgxMQ9kFgICAQ9kFgICAQ9kFgQCAQ8WBB4EaHJlZgl
  Form item: "__VIEWSTATEGENERATOR" = "C2EE9ABB"
  [...] Form item: "__EVENTVALIDATION" = "/wEWwWLoz/fGCgLS tq24BwK3jsrkBALtuvfLDQKC3IeGDAKR1ca+D
  Form item: "tbUsername" = "niranjanu"
    Key: tbUsername
    Value: niranjanu
  Form item: "tbPassword" = "Test@123"
    Key: tbPassword
    Value: Test@123
  Form item: "cbPersistCookie" = "on"
  Form item: "btnLogin" = "Login"
    Key: btnLogin
    Value: Login
```

Text item (text), 806 bytes

Figure 6: Captured Form Data

## 5. OBSERVATIONS & RESULTS

| Field Name | Captured Key | Extracted Value |
|------------|--------------|-----------------|
| Username   | uname        | niranjanu       |
| Password   | pass         | Test@123        |

## 6. PREVENTIVE COUNTERMEASURES

1. **Use HTTPS:** Encrypt client-server communication using TLS so that captured network traffic cannot directly reveal login credentials.
2. **Use HSTS:** HTTP Strict Transport Security (HSTS) helps prevent browsers from accessing websites through unencrypted HTTP.
3. **Network Security:** Use secure Wi-Fi, VLANs, and appropriate access controls to reduce unauthorized packet capture.
4. **Avoid Plain-Text Protocols:** Replace insecure protocols such as HTTP, FTP, and Telnet with encrypted alternatives.

## 7. RESULT

The network traffic was successfully captured and analyzed using Wireshark. HTTP GET and POST requests were identified using display filters, and the submitted form data was observed in the HTTP POST packet because the communication was not encrypted.